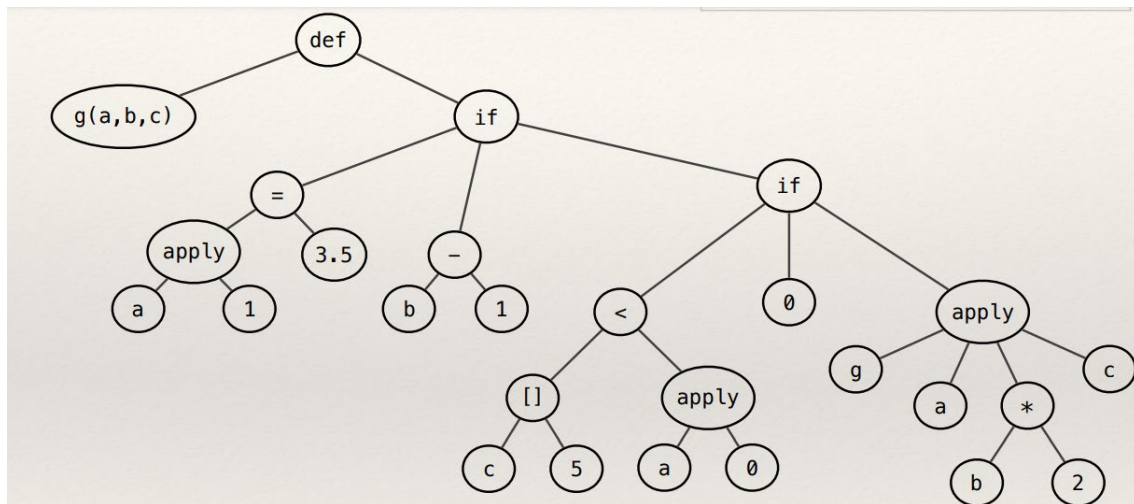


Actividad

Dado lo discutido y un ejemplo de código imperativo en el cual no se incluye información de tipos:

```
g(a,b,c) = if a(1) = 3.5 then
           b - 1
           else
             if c[5] < a(0) then
               0
             else
               g(a, b * 2, c)
```

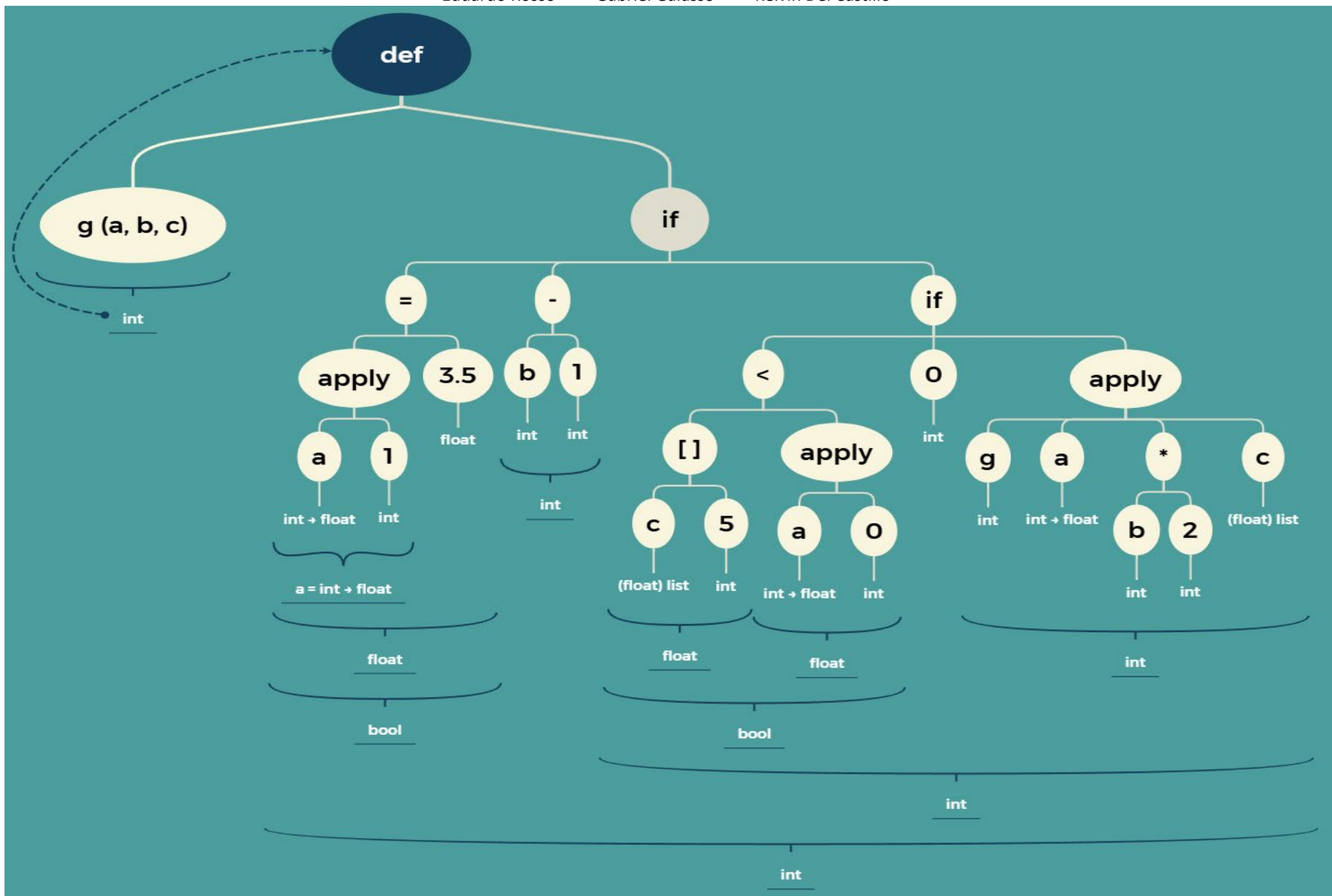
Considerando además el árbol de sintaxis abstracta del fragmento de código mostrado más arriba:



Se requiere que los estudiantes - siguiendo la heurística discutida en clases - hagan una exploración (ascendente o descendente) del árbol y encuentren los tipos de todas las variables y funciones que aparecen en el fragmento de código. Para esto deberán encontrar los tipos (simples o compuestos) de las variables de tipo que aparecen en la siguiente tabla:

Variable/ Función	Tipo
g	$\alpha \rightarrow \beta \Rightarrow \alpha = (\text{int} \rightarrow \text{float}) * \text{int} * \text{float list}$ $\text{y } \beta = \text{int}$
a	$\gamma = \text{int} \rightarrow \text{float}$
b	$\eta = \text{int}$
c	$\epsilon = \text{float list}$

1. El árbol rotulado con cada uno de los tipos correspondientes a los nodos.



2. Una breve explicación (1 párrafo, 6 líneas máximo) sobre el proceso que se siguió

Se tipó el árbol usando una exploración ascendente: primero se asignaron tipos a literales (3.5: float; 0,1,2,5: int) y operadores. De $a(1)=3.5$ se dedujo que $a(1):float$, por lo que $a:int \rightarrow float$. De $b-1$ y $b*2$ se obtuvo $b:int$. De $c[5] < a(0)$ se concluyó que $c[5]:float$, entonces $c:float\ list$. Finalmente, como en ambos condicionales las ramas deben tener el mismo tipo y aparecen $b-1$ y 0 (int), toda la función retorna int, quedando $g:(int \rightarrow float)*int*float\ list \rightarrow int$